task.sql

university_db/postgres@PostgreSQL 18

No limit

Query | Query History

```sql
-- Q1
SELECT
    student_id,
    first_name,
    last_name,
    COALESCE(nationality, 'Unknown') AS nationality
FROM students;

-- Q2
SELECT
    first_name,
    last_name,
    gpa AS original_gpa,
    NULLIF(gpa, 0.0) AS cleaned_gpa
FROM students;

-- Q3
SELECT
    first_name,
    last_name,
    COALESCE(NULLIF(gpa, 0.0), 'Not Evaluated') AS gpa_status
FROM students;
```

```sql

-- Bonus
SELECT
    d.dept_name,
    COUNT(s.student_id) AS student_count,
    COALESCE(
        SUM(s.gpa) / NULLIF(COUNT(s.gpa), 0),
        0
    ) AS avg_gpa
FROM departments d
LEFT JOIN students s ON d.dept_id = s.dept_id
GROUP BY d.dept_name;

-- Q4
CREATE TEMP TABLE temp_course_stats AS
SELECT
    c.course_code,
    c.course_name,
    COUNT(e.student_id) AS enrolled_count,
    AVG(e.grade) AS avg_grade
FROM courses c
LEFT JOIN enrollments e ON c.course_id = e.course_id
GROUP BY c.course_code, c.course_name;
SELECT *
FROM temp_course_stats
WHERE avg_grade > 75;

```

```sql
-- Q5
CREATE INDEX idx_students_dept_id
ON students(dept_id);

-- Q6
CREATE UNIQUE INDEX idx_students_email
ON students(email);
INSERT INTO students (first_name, last_name, email)
VALUES ('Mostafa', 'Shanab', 'mostafa.shanab@student.edu');

-- Q7
CREATE INDEX idx_prof_salary_active
ON professors(salary)
WHERE is_active = TRUE;

-- Q8
CREATE VIEW v_student_details AS
SELECT
    s.student_id,
    s.first_name || ' ' || s.last_name AS full_name,
    s.email,
    s.dept_id,
    s.gpa,
    d.dept_name,
    f.faculty_name
FROM students s
JOIN departments d ON s.dept_id = d.dept_id
JOIN faculties f ON d.faculty_id = f.faculty_id;
SELECT *
FROM v_student_details
WHERE dept_id = 3;
```

```sql
-- Q9
CREATE TABLE enrollment_audit (
    audit_id SERIAL PRIMARY KEY,
    student_id INT,
    old_grade NUMERIC,
    new_grade NUMERIC,
    changed_at TIMESTAMP DEFAULT NOW(),
    changed_by TEXT
);
CREATE OR REPLACE FUNCTION log_grade_change()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.grade IS DISTINCT FROM OLD.grade THEN
        INSERT INTO enrollment_audit(student_id, old_grade, new_grade, changed_by)
        VALUES (OLD.student_id, OLD.grade, NEW.grade, current_user);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_grade_update
BEFORE UPDATE ON enrollments
FOR EACH ROW
EXECUTE FUNCTION log_grade_change();

-- Q10
UPDATE enrollments
SET grade = 90
WHERE enrollment_id = 1;
select * from enrollment_audit;
```

```sql
-- Q11
CREATE OR REPLACE FUNCTION fix_salary()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.salary IS NULL OR NEW.salary < 5000 THEN
        NEW.salary := 5000;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_fix_salary
BEFORE INSERT ON professors
FOR EACH ROW
EXECUTE FUNCTION fix_salary();
INSERT INTO professors (first_name, last_name, email, dept_id, salary)
VALUES ('Test', 'Prof', 'test@uni.edu', 1, 1000);
select * from professors;
```

```sql
-- Q12
CREATE TABLE IF NOT EXISTS salary_log (
 log_id SERIAL PRIMARY KEY,
 prof_id INTEGER,
 old_salary NUMERIC,
 new_salary NUMERIC,
 changed_by TEXT DEFAULT CURRENT_USER,
 changed_at TIMESTAMPTZ DEFAULT NOW()
);
BEGIN;
WITH updated_salaries AS (
    UPDATE professors
    SET salary = salary * 1.10
    WHERE dept_id = 1
    RETURNING
        prof_id,
        salary / 1.10 AS old_salary,
        salary AS new_salary
)
INSERT INTO salary_log (prof_id, old_salary, new_salary)
SELECT
    prof_id,
    old_salary,
    new_salary
FROM updated_salaries;
SELECT * FROM professors WHERE dept_id = 1;
SELECT * FROM salary_log;
COMMIT;
```

```sql
-- Q13
BEGIN;
DELETE FROM enrollments
WHERE student_id = 1;
SELECT * FROM enrollments
WHERE student_id = 1;
ROLLBACK;
SELECT * FROM enrollments
WHERE student_id = 1;

-- Q14
BEGIN;

UPDATE faculties
SET budget = budget + 500000
WHERE faculty_id = 1;

SAVEPOINT after_first_update;

UPDATE faculties
SET budget = budget + 500000
WHERE faculty_id = 2;

ROLLBACK TO SAVEPOINT after_first_update;

COMMIT;
```

```sql
-- Q15
CREATE ROLE uni_readonly;
GRANT CONNECT ON DATABASE university_db TO uni_readonly;
GRANT USAGE ON SCHEMA public TO uni_readonly;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO uni_readonly;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
GRANT SELECT ON TABLES TO uni_readonly;
GRANT uni_readonly TO postgres;
SET ROLE uni_readonly;

SELECT * FROM students;

INSERT INTO students (first_name, last_name, email)
VALUES ('Test', 'User', 'test_readonly@student.edu');

RESET ROLE;
```

```sql
-- Q16
CREATE ROLE uni_readwrite;
GRANT CONNECT ON DATABASE university_db TO uni_readwrite;
GRANT USAGE ON SCHEMA public TO uni_readwrite;

GRANT SELECT, INSERT, UPDATE, DELETE
ON ALL TABLES IN SCHEMA public
TO uni_readwrite;
GRANT uni_readwrite TO postgres;

REVOKE DELETE ON students FROM uni_readwrite;

SET ROLE uni_readwrite;
DELETE FROM students WHERE student_id = 1;

RESET ROLE;

REVOKE ALL PRIVILEGES ON students FROM uni_readwrite;

SET ROLE uni_readwrite;

SELECT * FROM students;

-- Q17
-- pg_dump -U postgres -d university_db -F c -f full_backup.dump
-- pg_dump -U postgres -d university_db --schema-only -f schema_backup.sql
-- pg_dump -U postgres -d university_db --data-only -f data_backup.sql
```

Select Administrator: Command Prompt

```
Microsoft Windows [Version 10.0.26200.8039]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>cd C:\Program Files\PostgreSQL\18\bin

C:\Program Files\PostgreSQL\18\bin>pg_dump -U postgres -d university_db -F c -f full_backup.dump
Password:


C:\Program Files\PostgreSQL\18\bin>pg_dump -U postgres -d university_db -F c -f full_backup.dump
Password:


C:\Program Files\PostgreSQL\18\bin>pg_dump -U postgres -d university_db --schema-only -f schema_backup.sql
Password:

pg_dump: error: connection to server at "localhost" (::1), port 5432 failed: FATAL:  password authentication failed for
user "postgres"

C:\Program Files\PostgreSQL\18\bin>pg_dump -U postgres -d university_db --schema-only -f schema_backup.sql
Password:


C:\Program Files\PostgreSQL\18\bin>pg_dump -U postgres -d university_db --schema-only -f schema_backup.sql
Password:


C:\Program Files\PostgreSQL\18\bin>pg_dump -U postgres -d university_db --data-only -f data_backup.sql
Password:

pg_dump: warning: there are circular foreign-key constraints on this table:
pg_dump: detail: professors
pg_dump: hint: You might not be able to restore the dump without using --disable-triggers or temporarily dropping the co
nstraints.
pg_dump: hint: Consider using a full dump instead of a --data-only dump to avoid this problem.

C:\Program Files\PostgreSQL\18\bin>
```

This PC > Local Disk (C:) > Program Files > PostgreSQL > 18 > bin

Search bin

Sort View ...

Name	Date modified	Type	Size
data_backup.sql	2026-03-26 4:31 PM	SQL Source File	19 KB
schema_backup.sql	2026-03-26 4:30 PM	SQL Source File	25 KB
full_backup.dump	2026-03-26 4:29 PM	DUMP File	44 KB
clusterdb.exe	2026-03-05 2:36 PM	Application	131 KB
createdb.exe	2026-03-05 2:36 PM	Application	129 KB
createuser.exe	2026-03-05 2:36 PM	Application	134 KB
dropdb.exe	2026-03-05 2:36 PM	Application	128 KB
dropuser.exe	2026-03-05 2:36 PM	Application	128 KB
ecpg.exe	2026-03-05 2:36 PM	Application	908 KB
icudt77.dll	2026-03-05 2:36 PM	Application extens...	31,151 KB
icuin77.dll	2026-03-05 2:36 PM	Application extens...	3,501 KB
icuio77.dll	2026-03-05 2:36 PM	Application extens...	62 KB
icutu77.dll	2026-03-05 2:36 PM	Application extens...	373 KB
icuuc77.dll	2026-03-05 2:36 PM	Application extens...	2,493 KB
initdb.exe	2026-03-05 2:36 PM	Application	238 KB
libcrypto-3-x64.dll	2026-03-05 2:36 PM	Application extens...	5,565 KB
libcurl.dll	2026-03-05 2:36 PM	Application extens...	658 KB
libecpg.dll	2026-03-05 2:36 PM	Application extens...	100 KB
libecpg_compat.dll	2026-03-05 2:36 PM	Application extens...	25 KB
libiconv-2.dll	2026-03-05 2:36 PM	Application extens...	1,403 KB
libintl-9.dll	2026-03-05 2:36 PM	Application extens...	465 KB